

Medallion Architecture

Data Integration Project Report

Student: Willian Franco da Nóbrega

Student ID: GH1035092

GitHub: https://github.com/willianfnobrega/Data_Integration/tree/main

Data Set: <https://data.bettergov.ph/datasets/21>;

<https://www.census.gov/foreign-trade/schedules/b/index.html>;

<https://www.iban.com/country-codes>;

<https://help.adjust.com/en/article/countries-by-region>

Contents

1	Introduction	2
1.1	Business Problem	2
1.2	Objectives	2
1.3	Dataset	3
2	Project Architecture	8
2.1	Medallion Architecture	8
2.2	Bronze Layer	8
2.3	Silver Layer	9
2.4	Gold Layer	10
3	Results	14
4	Challenges	15
5	References	16

Chapter 1

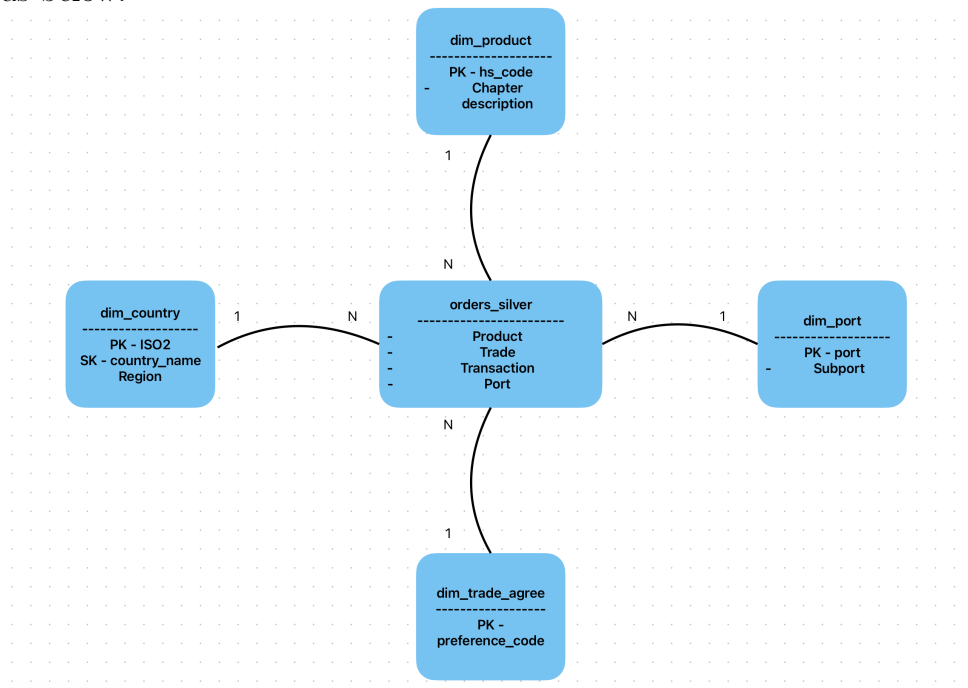
Introduction

1.1 Business Problem

The company requested us to assist in preparing their data for the analysts to use, the challenges they had is their data is overly complicated for the analysts to run simple queries and prepare quick dashboards.

For such reason, it was decided to create a Star Schema based on the orders details provided by the company, as well as other sources available.

From that table we will extract as much information as possible to create a Star Schema as below:



Besides the schema we also work with a layer system, where we have bronze, silver and golden layer, which will be explained further.

1.2 Objectives

The advantages of this schema are:

- Make queries and aggregations efficient and intuitive.
- Works great with BI tools.
- Business Friendly for analysts.

Although highly used, some challenges come with this type of schema:

- Less flexible when the information in the dimensional table changes often. It may become harder to keep historical data the same as well.
- Not very compatible with deep hierarchical systems.
- The schema is mostly build for analytics, not much for operational use.

1.3 Dataset

For this project we used 4 sources:

- Orders Details: This dataset contained around 65 million rows with the orders available. What was noticed is the data provided had many inconsistencies and gaps in information.

First: We updated the currencies values where we had "US8" and "US3" for example to "USD".

```
%sql
UPDATE business_catalog.operations_philippines.orders_silver
SET currency = 'USD'
WHERE currency RLIKE '(?i)^US[0-9]+$';
```

> [See performance \(1\)](#)

> `_sqlidf: pyspark.sql.connect.dataframe.DataFrame = [num_affected_rows: long]`

Table	
	num_affected_rows
1	5088

Then we: Identified and separated the repetitive rows, where all information is identical (very unlikely for such a detailed operational dataset). Those repetitive rows need to be further assessed by the transactions team to understand.

```

orders_silver_df = spark.table("orders_silver")
total_rows = orders_silver_df.count()
distinct_rows = orders_silver_df.distinct().count()

print(f"Duplicate rows: {total_rows - distinct_rows}")

duplicates = (
    orders_silver_df.groupBy(orders_silver_df.columns)
    .count()
    .filter("count > 1")
)

display(duplicates)

# Save duplicate rows to repetitive_rows table
duplicates.drop("count").write.mode("overwrite").saveAsTable(["business_catalog",
operations_philippines.repetitive_rows"])

# Remove duplicate rows from orders_silver and update the table
deduped_df = orders_silver_df.distinct()
deduped_df.write.mode("overwrite").saveAsTable("orders_silver")

```

> [See performance \(5\)](#) [Optimize](#)

> orders_silver_df: pyspark.sql.connect.dataframe.DataFrame = [year: integer, month: integer ... 35 more fields]
> duplicates: pyspark.sql.connect.dataframe.DataFrame = [year: integer, month: integer ... 36 more fields]
> deduped_df: pyspark.sql.connect.dataframe.DataFrame = [year: integer, month: integer ... 35 more fields]

Duplicate rows: 860920

	year	month	quarter	month_code	uid	entry_number	hs_code
1	2022	11	null	null	null	null	33021090

Next we had to fill the quarter column based on months information:

```

%sql
UPDATE orders_silver
SET quarter = CASE
    WHEN month <= 3 THEN 1
    WHEN month > 3 AND month <= 6 THEN 2
    WHEN month > 6 AND month <= 9 THEN 3
    WHEN month > 9 AND month <= 12 THEN 4
    ELSE quarter
END
WHERE quarter IS NULL AND month IS NOT NULL;

```

> [See performance \(1\)](#) [Optimize](#)

> _sql_df: pyspark.sql.connect.dataframe.DataFrame = [num_affected_rows: long]

	num_affected_rows
1	44047188

It was also projected to run a validation check-up on the tables such as the example below, but the idea was suspended for the time being since no invalid columns were found.

The idea below is to identify columns with number figures that should not be negative but, for some error in the system (based on other experiences), ended up as negative:

```

31 Python
# Get all double columns
double_columns = [
    field.name
    for field in orders_silver_df.schema.fields
    if field.dataType.simpleString() == "double"
]

# Start with a spark column full of False (no negative values)
negative_condition = F.lit(False)

# Check each double column
for col_name in double_columns:
    negative_condition = negative_condition | (F.col(col_name) < 0)

validated_df = orders_silver_df.withColumn(
    "Validation_Status",
    F.when(negative_condition, "Negative Values")
    .otherwise("Ok")
)

```

The idea was discarded for the time being as we could not identify any invalid examples.

- HS Chapter: This dataset is used to provide us the description of the HS codes we have and no cleaning was needed besides setting up the header. It was included in our dimensional product table as below:

```

dim_product = (
    df.select("hs_code", "goods_description", "unit_of_measure")
    .dropDuplicates(["hs_code"])
    .filter(F.col("hs_code").isNotNull())
    .withColumn("hs_chapter", F.col("hs_code").substr(1, 2))
    .withColumn("hs_heading", F.col("hs_code").substr(1, 4))
    .withColumn("hs_subheading", F.col("hs_code").substr(1, 6))
)

dim_product = dim_product.join(
    chapters_hs_df, on="hs_chapter", how="left"
)

display(dim_product)

```

> [See performance \(1\)](#) ✓ Checked for improvement

> dim_product: pyspark.sql.connect.dataframe.DataFrame = [hs_chapter: string, hs_code: string ... 5 more fields]

	hs_chapter	hs_code	goods_description
1	84	84799030000	SPACER RING
2	85	85182990000	SPEAKER
3	84	84834030000	2EA DRIVE GEAR SHAFT, 2EA GEAR SHAFT
4	84	84242029000	HOSE REEL
5	61	61152100000	556.00 PC GIRL TIGHTS KNITTED 92%PA
6	52	52052300000	NE 30/1 100% COTTON YARN

We kept all codes from our order details df as we do not find the need to store the codes not used.

- Country Code: This dataset was used for us to quickly create our country tables based on the origin and export codes, the origin column used ISO3 and the export column used Country Name, so we had to use both keys to identify the naming and bring the country description standardized for all cases.

We had to first unify both sets (origin and export) with exclusive values only, and then we applied the join.

```
#unify both sets
dim_country = (
    countries_origin.select("country_code")
    .union(
        countries_export.select("country_code")
    )
    .dropDuplicates(["country_code"])
)

country_dict = {}

# Build a dictionary mapping both country names and Alpha-2 codes to Alpha-3 codes
for row in countries_code.select(
    "Country",
    "Alpha-2 code",
    "Alpha-3 code"
).collect():
    if row["Country"]:
        country_dict[row["Country"]] = row["Alpha-3 code"]
    if row["Alpha-2 code"]:
        country_dict[row["Alpha-2 code"]] = row["Alpha-3 code"]

# build a F.when() expression to replace country_code with the corresponding Alpha-3 code
expr = F.col("country_code")
for key, iso3 in country_dict.items():
    expr = F.when(
        F.col("country_code") == key,
        iso3
    ).otherwise(expr)

dim_country = dim_country.withColumn(
    "country_code",
    expr
)

# add country name
dim_country = (
    dim_country
    .join(
        countries_code.select(
            F.col("Country").alias("country_name"),
            F.col("Alpha-3 code").alias("country_code"),
            F.col("Alpha-2 code").alias("ISO2")
        ),
        on="country_code",
        how="left"
    )
    .dropDuplicates(["country_code"])
)

dim_country = (
    dim_country.filter(F.col("country_name").isNotNull())
)

display(dim_country)

> See performance (2)

> dim_country: pyspark.sql.connect.dataframe.DataFrame = [country_code: string, country_name: string ... 1 more field]
```

	country_code	country_name	ISO2
1	UGA	Uganda	UG
2	HND	Honduras	HN
3	BGD	Bangladesh	BD

Figure 1.1: Country standardization process and the resulting Country dimension table.

- Countries and Regions: This dataset was used to quickly add the region of each country to our dim country, based on the ISO2 key.

```
df_regions = (
    spark.read.format("excel")
        .option("header", "false")
        .option("inferSchema", "true")
        .load("/Volumes/business_catalog/operations_philippines/main_data/Regions_Countries.xlsx")
)

header_row = df_regions.first()
column_names = [header_row[i] for i in range(len(df_regions.columns))]

df_regions = df_regions.toDF(*column_names).filter(F.col(column_names[0]) != column_names[0])

df_regions = df_regions.withColumn("Country code", F.upper(F.col("Country code")))

display(df_regions)
```

> [See performance \(2\)](#)

> df_regions: pyspark.sql.connect.dataframe.DataFrame = [Country code: string, Country: string ... 1 more field]

	Country code	Country	Region
1	AD	Andorra	EMEA
2	AE	United Arab Emirates	EMEA

Chapter 2

Project Architecture

2.1 Medallion Architecture

We decided to work the medallion architecture here because of the benefits for data analysts, we understand that the company had to organize their uploading workflow in a way that: they can import whatever they receive without the need of cleaning and assessing the data too much (bronze layer), have an area where the data is clean, but still in technical format so the data department can easily customize their needs and requests providing a full view of what is available (silver layer) and having an easy layer for analysts to build their reports, dashboards and have insights on the data already cleaned.

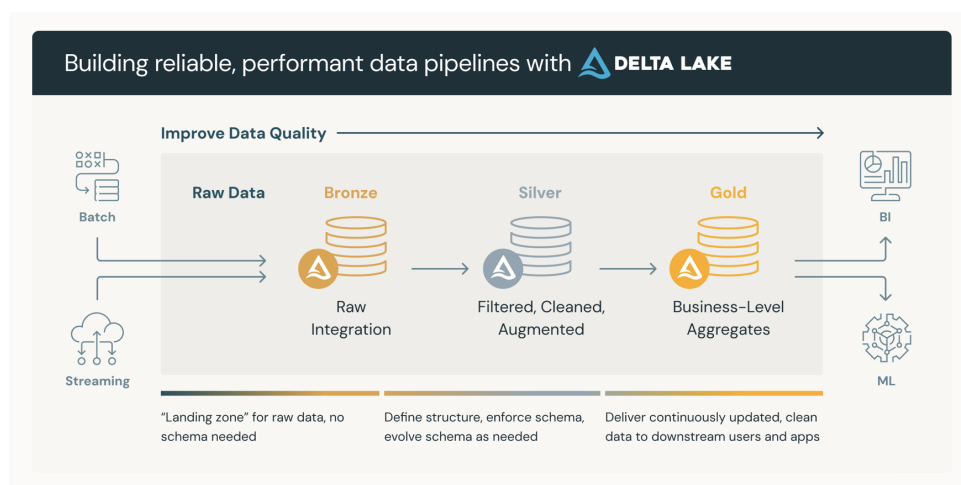


Figure 2.1: Image from: What is a Medallion Architecture? - Databricks

5

2.2 Bronze Layer

The bronze layer provides us with a quick historical data archive, the main point is the partition we do by year so we can observe each year historical data easily in case any assessment is needed in the future.

8

Writing the table in the data lake so we can have extra safety in case of corruption or other issues with the data ingestion

We will partition per year for organization purposes.

```

(
  orders_df.write
    .format("delta")
    .mode("overwrite")
    .option("mergeSchema", "true")
    .partitionBy("year")
    .saveAsTable(
      "business_catalog.operations_philippines.orders_bronze"
    )
)

```

> [See performance \(1\)](#) ✓ Checked for improvements

```
%sql
SELECT *
FROM business_catalog.operations_philippines.orders_bronze
LIMIT 10;
```

> [See performance \(1\)](#) Optimize

> `_sqldf: pyspark.sql.connect.dataframe.DataFrame = [year: integer, month: integer ... 35 more fields]`

	i_3 year	i_3 month	i_3 quarter	A_0 month_code	A_0 uid	A_0 entry_number	A_0 hs_code	A_0 goods_description
1	2023		4	null null	null	null	40103900000	BELT 50 AT15/2745 BFX S
2	2023		4	null null	null	null	40103900000	BELT 16 AT10/1860 SYN S
3	2023		4	null null	null	null	84779010000	AIR ROTARY UNION 40 7B
4	2023		4	null null	null	null	84789000000	1 PACKAGE SPARE PARTS
5	2023		4	null null	null	null	84835000000	1 PC. TOOTHED BELT PUL
6	2023		4	null null	null	null	30049099000	PHARMACEUTICAL PROD
7	2023		4	null null	null	null	33030000000	LOUIS VUITTON PERFUME
8	2023		4	null null	null	null	33030000000	LOUIS VUITTON PERFUME
9	2023		4	null null	null	null	33030000000	LOUIS VUITTON PERFUME
10	2023		4	null null	null	null	33030000000	LOUIS VUITTON PERFUME

Figure 2.2: Bronze Table Creation

2.3 Silver Layer

Here the data is cleaned and matched enough so we can create an enterprise view for users.

This layer brings data from other sources as well such as country descriptions and the chapters mentioned before.

All this information so we can match and have our dimensional table more complete. This layer can be used mostly by data technical users who have a machine learning model to experiment for example.

```
spark.sql("""
CREATE OR REPLACE TABLE business_catalog.operations_philippines.dim_product AS
SELECT * FROM dim_product
""")

spark.sql("""
CREATE OR REPLACE TABLE business_catalog.operations_philippines.dim_country AS
SELECT * FROM dim_country
""")

spark.sql("""
CREATE OR REPLACE TABLE business_catalog.operations_philippines.dim_port AS
SELECT * FROM dim_port
""")

spark.sql("""
CREATE OR REPLACE TABLE business_catalog.operations_philippines.dim_trade_agree AS
SELECT * FROM dim_trade_agree
""")

> View performance \(4\) ✓ Checked for improvement
DataFramesum.affected_rows: bigint, num_inspected_rows: bigint
```

Figure 2.3: Silver Tables Creation

A problem we have in the quality of our dimensional tables is no description for the trade agreements were found available for us to have more insights on it. As well as the HS chapters could be grouped together in example: chapter 1-10 (Agriculture related).

2.4 Gold Layer

This layer is used mostly for specific reports desired by the stakeholders, so we have already cleaned data that is joined and grouped, as well as used for calculations, that together creates a flexible straightforward environment for insights.

Used for business cases.

In our project we created:

- Revenue by Production and Year: Show us a resume of revenues per product, year and quarter.
- Top Trading Partners by Year: Show us a resume of the partners we have been acquiring products from and the benefit from it in revenue, grouped by country, region and year.
- Port Performance: Show us the performance of the Ports in a revenue based, grouped by port, subport and year.
- FTA Utilization: Show us which Fair Trade Agreements are benefiting us the most, grouped by preference code, year and the totals. Here we did 2 joins so we can run a calculation operation per year and aggregate the values from it. The inner join is because we want to remove the operations with no agreements, since we already know they will be always higher than the ones with agreements.

```
%sql
CREATE OR REPLACE TABLE revenue_per_product_chapter_year AS
SELECT
    chapter_description,
    s.year,
    s.quarter,
    SUM(s.dutiable_value_php) AS `Total_Dut_V_PHP`,
    SUM(s.duties_taxes_total) AS `Total_Dut_Taxes`,
    SUM(s.cif_value) AS `Total_CIF_USD`,
    COUNT(*) AS `Transaction_Count`, -- Had to count all rows like this since the IDs are incomplete.
    SUM(s.quantity) AS `Total_Quantity`
FROM orders_silver s
LEFT JOIN dim_product p
USING (hs_code)
GROUP BY
    chapter_description,
    s.year,
    s.quarter;
```

Figure 2.4: Revenue by Production and Year

```
%sql
-- Top Trading Partners by Year
CREATE OR REPLACE TABLE revenue_per_country_year AS
SELECT
    c.ISO2,
    c.country_name,
    c.Region,
    s.year,
    SUM(s.dutiable_value_php) AS `Total_Dut_V_PHP`,
    SUM(s.duties_taxes_total) AS `Total_Dut_Taxes`,
    COUNT(*) AS `Transaction_Count` -- Had to count all rows like this since the IDs are incomplete.
FROM orders_silver s
LEFT JOIN dim_country c
ON s.country_origin = c.ISO2
GROUP BY
    c.ISO2,
    c.country_name,
    c.Region,
    s.year;
```

Figure 2.5: Top Trading Partners by Year

```
%sql
-- Port Performance
CREATE OR REPLACE TABLE port_performance AS
SELECT
    s.port,
    s.subport,
    s.year,
    SUM(s.dutiable_value_php) AS `Total_Dut_V_PHP`,
    SUM(s.duties_taxes_total) AS `Total_Dut_Taxes`,
    COUNT(*) AS `Shipments`
FROM orders_silver s
LEFT JOIN dim_port p
ON s.port = p.port
WHERE s.port IS NOT NULL
GROUP BY
    s.port,
    s.subport,
    s.year;
```

Figure 2.6: Port Performance

```

-- Free Trade Agreement Benefits
CREATE OR REPLACE TABLE free_trade_agreement_benefits AS
SELECT
    t.preference_code,
    s.year,
    COUNT(*) AS 'Transactions_Using_FTA',
    SUM(s.duty) AS 'Duty_Paid',
    SUM(s.dutiable_value_php) AS 'Total_Dutiable_Value',
    ROUND(
        SUM(s.dutiable_value_php) / y.year_total * 100,
        2
    ) AS 'Dutiable_Value_%_of_Year'
FROM orders_silver s
-- We do an Inner Join here so we keep only the codes already filtered in our Dim_Table that are also being used in the silver
main table.
INNER JOIN dim_trade_agree t
    ON s.preference_code = t.preference_code
-- We have to create a double join here so we can have the total per year calculated
JOIN (
    SELECT
        year,
        SUM(dutiable_value_php) AS year_total
    FROM orders_silver
    WHERE preference_code IS NOT NULL
    GROUP BY year
) y
    ON s.year = y.year
GROUP BY
    t.preference_code,
    s.year,
    y.year_total;

```

Figure 2.7: FTA Utilization

SQL

SELECT * FROM revenue_per_product_chapter_year LIMIT 10;

See performance (1)

Table

	1. preference_code	2. year	3. Transactions_Using_FTA	4. Duty_Paid	5. Total_Dutiable_Value	6. Dutiable_Value_%_of_Year
1	AFTA	2016	110806	1116675759	625174380788	48.86
2	ACFTA	2016	270794	1442346967	406759645975	31.79
3	AKFTA	2016	9601	1066133600	50685489474	3.96
4	JPEPA	2016	8460	1596826593	78991978736	6.17
5	ANFTA	2016	11335	151131180	54500394425	4.26
6	BOI	2016	3001	31706575	45757955447	3.58
7	AFMA	2016	225	7729632	1624569216	0.13
8	AIFTA	2016	4557	642558443	13034017150	1.02
9	AJCEP	2016	503	3256980	2802417575	0.22
10	AICO	2016	23	983902	71879590	0.01

10 rows | 2.313s runtime

Refreshed 13 minutes ago

This result is stored as _sql.gdf and can be used in other Python and SQL cells.

Send feedback

02:54 PM (2s)

SQL

SELECT * FROM revenue_per_country_year LIMIT 10;

See performance (1)

Table

	1. ISO2	2. country_name	3. Region	4. year	5. Total_Dut_Y_PHP	6. Total_Dut_Taxes	7. Transaction_Count
1	RU	Russia	EMEA	2016	5230733532087	440325749639	3140436
2	HK	Hong Kong	APAC	2022	181243767951.8584	23510311376.800083	201450
3	AE	United Arab Emirates (U...	EMEA	2024	136285463890.55014	29769377786.16004	18347
4	SE	Sweden	EMEA	2022	10232394498.39001	1747335608.050003	23027
5	GE	Georgia	EMEA	2022	336337754.9699999	69627426.16999999	376
6	CR	Costa Rica	LATAM	2022	1109832269.3000023	163293951.01	4286
7	NL	Netherlands (the)	EMEA	2024	32967205198.62007	4784943635.59	44545
8	AZ	Azerbaijan	EMEA	2022	1418034.4999999998	183603.88999999998	5
9	CX	Christmas Island	APAC	2022	3492587.54	651007.22	7
10	TT	Trinidad and Tobago	LATAM	2024	1120972.11	528374.05	29

Figure 2.8: Gold Tables 1 and 2, Head

```

%sql
SELECT * FROM port_performance LIMIT 10;

```

[See performance \(1\)](#)
✓ Checked for improvements

	port	subport	year	1.2 Total_Dut_V_PHP	1.2 Total_Dut_Taxes	1.2 Shipments
1	Port of Aparit	Sub-Port of Curimaao	2016	525522845	70255791	29
2	Port of Batangas	Sub-Port of Puerto Princ...	2016	6438256191	88906216	160
3	Port of Surigao	Sub-Port of Bislig	2016	9110459	1118814	1
4	Port of Batangas	Port of Batangas	2016	478008334878	76769625248	37291
5	Port of Cagayan de ...	Port of Cagayan de Oro	2016	25717796646	2713783397	1159
6	Port of Limay	Sub-Port of Mariveles	2016	76452305937	11441691573	1052
7	Port of San Fernando	Port of San Fernando	2016	5691205369	447914672	944
8	Port of Davao	Sub-Port of Parang	2016	110492654	13153681	12
9	Port of Manila	Sub-Port of Masinloc	2016	2441678328	378856118	8
10	Port of Manila	Port of Manila	2016	407301816445	47941018998	412192

10 rows | 1.509s runtime
Refreshed 13 minutes ago

This result is stored as `_sqlidf` and can be used in other [Python](#) and [SQL](#) cells.
Send feedback

02:55 PM (1s)
13

```

%sql
SELECT * FROM free_trade_agreement_benefits LIMIT 10;

```

[See performance \(1\)](#)
✓ Checked for improvements

```

%sqlidf: pyspark.sql.connect.dataframe.DataFrame = [preference_code: string, year: integer ... 4 more fields]

```

	preference_code	year	Transactions_Using_FTA	1.2 Duty_Paid	1.2 Total_Dutiable_Value	1.2 Dutiable_Value_%_of_Year
1	AFTA	2016	110806	1116675759	625174380788	48.86
2	ACFTA	2016	270794	1442346987	406759645975	31.79
3	AKFTA	2016	9601	1066133600	50685489474	3.96
4	JPEPA	2016	8460	1596826593	78991978738	6.17
5	ANFTA	2016	11335	151131180	54500394425	4.26
6	BOI	2016	3001	31706575	45797955447	3.58
7	AFMA	2016	225	7729632	1624669216	0.13
8	AIFTA	2016	4557	642558443	13034017150	1.02
9	AJCEP	2016	503	3266980	2802417575	0.22
10	INCO	2016	23	863302	716726530	0.01

Figure 2.9: Gold Tables 3 and 4, Head

Chapter 3

Results

The data was challenging to resume, deal with the gaps and create the codes for the dimensional tables.

But it was possible in the end to create the golden tables desired for the dashboards needed and implement the Star Schema requested as long as a medallion architecture.

A subject that needs attention are the many repetitive rows identified and separated in the repetitive rows table. They need to be reviewed to confirm if it was a system error, human error or if they are indeed identical operations in the same time and day. It is interesting to understand why it happens and mitigate for future uploads.

For the future, it is possible now to connect our tables to a provider such as Power BI to make those insights readable for the average user.

Alongside, a throughout validation on the orders file, such as the idea proposed earlier, will help to provide a data with higher quality, avoiding also costs in fixing it.

Chapter 4

Challenges

Main problem faced was the gap of information and descriptions available. For many of our dimensional tables, the solution was to bring external data sources validated to fill the lack of descriptions. It solves the problem for the time being, but for the future, a better maintenance has to be done for the database.

Another challenge was the lack of Operation ID, a critical key that could have prevented the existence of duplicated rows in our system.

Human entries also made the job harder than it should be, since currencies for example contained typos such as "US8". An implementation of automation in currencies would solve the issue, to avoid users to type information as much as possible.

Chapter 5

References

References

- Databricks (2026) *What is the Medallion Lakehouse Architecture?* Available at: <https://www.databricks.com/blog/what-is-medallion-architecture> (Accessed: 26 June 2026).
- BetterGovPH (2025) *Open Customs Data*. Hugging Face. Available at: <https://huggingface.co/datasets/bettergovph/open-customs-data> (Accessed: 26 June 2026).
- U.S. Census Bureau (2026) *Schedule B Search and Classification*. Available at: <https://www.census.gov/foreign-trade/schedules/b/index.html> (Accessed: 26 June 2026).
- IBAN.com (2026) *Country Codes*. Available at: <https://www.iban.com/country-codes> (Accessed: 26 June 2026).
- Adjust GmbH (2026) *Countries by Region*. Available at: <https://help.adjust.com/en/article/countries-by-region> (Accessed: 26 June 2026).